Final Project: Complete Time Series Analysis, Benchmarking & Ensemble Forecasting

1. Setup & Package Installation

```
# Install required forecasting & analysis packages
!pip install -q statsmodels statsforecast==2.1.1 neuralforecast==3.3.0 chronos-forecasting==2.3.2 "timesfm[tc
```

```
import warnings
warnings.filterwarnings("ignore")
from pathlib import Path
import os
import time
import hashlib
from statistics import NormalDist

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats

from statsmodels.tsa.seasonal import STL, MSTL as StatsmodelsMSTL, seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.tsa.stattools import kpss
from statsforecast.arima import ndiffs, nsdiffs
from statsforecast.models import AutoARIMA, AutoETS, ARIMA, MSTL

import torch
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"PyTorch device: {DEVICE}")
```

```
PyTorch device: cuda
```

2. Load Data & Calendar Interrogation

```
SURNAME = "KAPRANOV"
MY_ID   = "mkapranov@kse.org.ua"
M       = 24
H       = 48
LEVEL   = 95
ALPHA   = 1 - LEVEL / 100
LO, HI  = f"lo-{LEVEL}", f"hi-{LEVEL}"
Z_LEVEL = NormalDist().inv_cdf(1 - ALPHA / 2)
Z_80    = NormalDist().inv_cdf(0.90)

EXOG = ["cloud_cover", "temperature", "wind_speed", "pressure", "humidity", "daylight", "air_density"]

train_path = Path("train.csv")
weather_path = Path("weather_forecast.csv")

if not train_path.exists():
    raise FileNotFoundError("train.csv not found!")

raw_train = pd.read_csv(train_path)
weather_fc = pd.read_csv(weather_path, parse_dates=["ds"]) if weather_path.exists() else None
```

```
def interrogate_and_repair_calendar(df: pd.DataFrame, freq: str = "h", m: int = 24, ds_col: str = "ds", y_col
    extra_cols = [c for c in df.columns if c not in (ds_col, y_col)]
    out = df[[ds_col, y_col] + extra_cols].copy().rename(columns={ds_col: "ds", y_col: "y"})
    out["ds"] = pd.to_datetime(out["ds"])
    out = out.sort_values("ds").reset_index(drop=True)

    print("=== CALENDAR INTERROGATION ===")
    print(f"Rows:          {len(out)}")
```

```
    print(f"Span:          {out['ds'].min()} -> {out['ds'].max()} ({len(out) / m:.1f} cycles of m={m})")
    print(f"Missing vals:  {out.isna().sum().to_dict()}")
    print(f"Duplicates:    {out['ds'].duplicated().sum()}")
    print(f"y Min / Mean / Median / Max / Std: {out['y'].min():.2f} / {out['y'].mean():.2f} / {out['y'].media
    print(f"Relative step-to-step noise: {np.std(np.diff(out['y'].dropna())) / out['y'].mean():.3f}")

    if out["ds"].duplicated().any():
        out = out.groupby("ds", as_index=False).mean(numeric_only=True)
        print(f"-> Resolved duplicates by averaging: {len(out)} rows")

    grid = pd.date_range(out["ds"].min(), out["ds"].max(), freq=freq)
    reindexed = out.set_index("ds").reindex(grid).rename_axis("ds")
    revealed_gaps = reindexed["y"].isna().sum() - out["y"].isna().sum()
    print(f"-> Regular grid ({freq}): {len(out)} rows -> {len(reindexed)} rows (revealed gaps: {revealed_gaps

    reindexed["y"] = reindexed["y"].interpolate(limit_area="inside")
    reindexed = reindexed.dropna(subset=["y"]).reset_index()
    return reindexed

data = interrogate_and_repair_calendar(raw_train, freq="h", m=M)
y = data["y"].to_numpy(float)
dates = data["ds"]

FUTURE = pd.DatetimeIndex(weather_fc["ds"]) if weather_fc is not None else pd.date_range(dates.iloc[-1] + pd.
X_ALL = np.vstack([data[EXOG].to_numpy(float), weather_fc[EXOG].to_numpy(float)]) if weather_fc is not None e
DS_ALL = pd.DatetimeIndex(pd.concat([dates, weather_fc["ds"]], ignore_index=True)) if weather_fc is not None
```

```
=== CALENDAR INTERROGATION ===
Rows:          33480
Span:          2023-01-01 01:00:00 -> 2026-10-27 00:00:00 (1395.0 cycles of m=24)
Missing vals:  {'ds': 0, 'y': 0, 'cloud_cover': 0, 'temperature': 0, 'wind_speed': 0, 'pressure': 0, 'humidity
Duplicates:    0
y Min / Mean / Median / Max / Std: 0.70 / 315.76 / 220.60 / 3111.30 / 377.60
Relative step-to-step noise: 0.323
-> Regular grid (h): 33480 rows -> 33480 rows (revealed gaps: 0)
```

3. Time Series Plot & Variance Stabilization

```
def swing_against_level(values: np.ndarray, ds: pd.Series, m: int = 24) -> pd.DataFrame:
    s_ds = pd.to_datetime(ds)
    group_key = s_ds.dt.to_period("M").astype(str) if m == 24 else s_ds.dt.year
    frame = pd.DataFrame({"group": group_key, "y": np.asarray(values, float)})
    return frame.groupby("group")["y"].agg(level="mean", swing=lambda v: v.max() - v.min(), std="std")


def plot_series_and_variance(df: pd.DataFrame, m: int = 24, label: str = "Electricity Generation (y)"):
    s_ds = pd.to_datetime(df["ds"])
    y_vals = df["y"].to_numpy(float)

    fig, axes = plt.subplots(3, 1, figsize=(15, 9))
    axes[0].plot(s_ds, y_vals, color="#1f77b4", linewidth=0.7)
    axes[0].set_title(f"{label} — Full History ({len(y_vals)} hours)")
    axes[0].set_ylabel("y")
    axes[0].grid(True, linestyle=":", alpha=0.5)

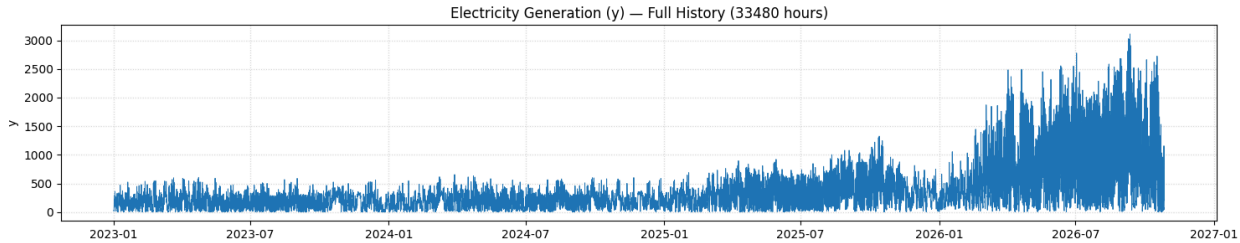
    zoom_n = min(len(y_vals), 14 * m)
    axes[1].plot(s_ds.iloc[-zoom_n:], y_vals[-zoom_n:], color="#1f77b4", linewidth=1.2, marker="o", markersiz
    axes[1].set_title(f"{label} — Last 14 Days (Hourly Profile)")
    axes[1].set_ylabel("y")
    axes[1].grid(True, linestyle=":", alpha=0.5)

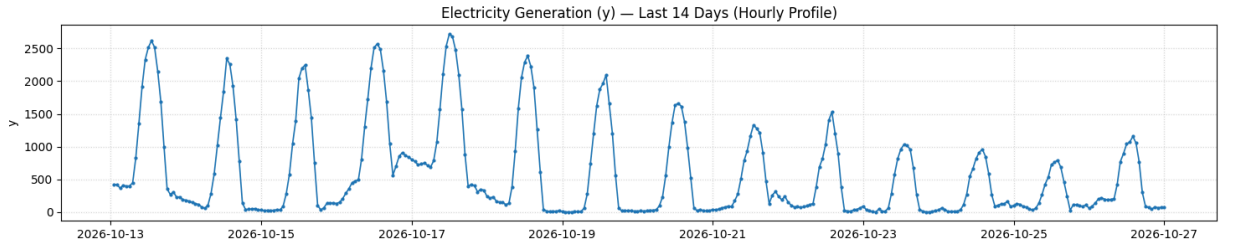
    log_y = np.log(np.maximum(y_vals, 1e-6))
    axes[2].plot(s_ds, log_y, color="#2ca02c", linewidth=0.7)
    axes[2].set_title(f"Log Transformed {label} (log(y))")
    axes[2].set_ylabel("log(y)")
    axes[2].grid(True, linestyle=":", alpha=0.5)
    plt.tight_layout()
    plt.show()

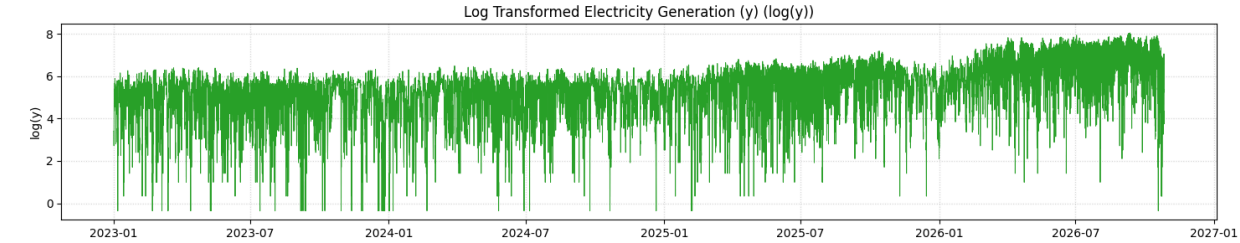
    sal = swing_against_level(y_vals, s_ds, m=m)
    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.scatter(sal["level"], sal["swing"], color="#1f77b4", alpha=0.8, edgecolors="k", s=45)
    r_val, p_val = stats.pearsonr(sal["level"], sal["swing"])
    m_slope, b_intercept = np.polyfit(sal["level"], sal["swing"], 1)
    ax.plot(sal["level"], m_slope * sal["level"] + b_intercept, color="crimson", linestyle="--",
            label=f"Fit (r = {r_val:.3f}, p = {p_val:.3e})")
    ax.set_title(f"Seasonal Swing vs. Level (Per Month, m={m})")
```

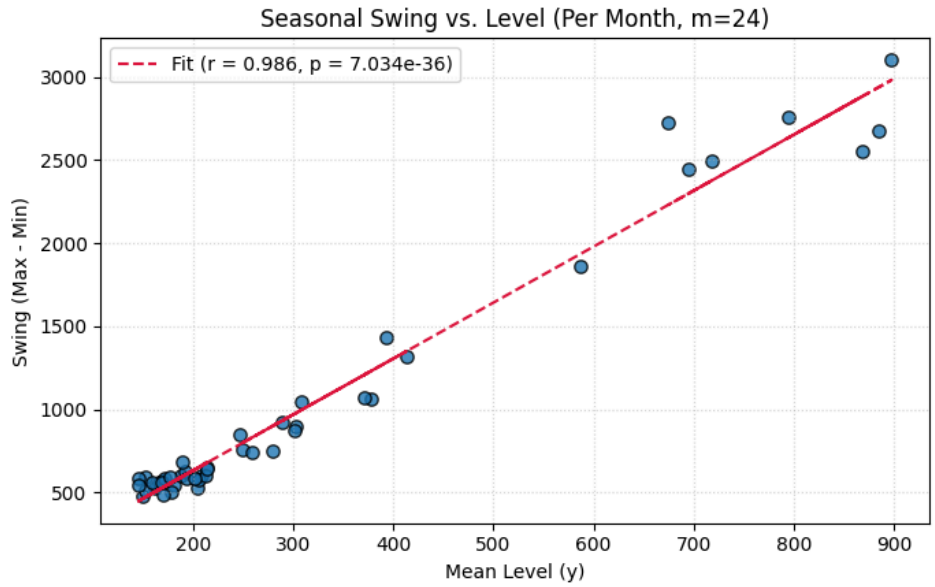
```
    ax.set_xlabel("Mean Level (y)")
    ax.set_ylabel("Swing (Max - Min)")
    ax.legend()
    ax.grid(True, linestyle=":", alpha=0.5)
    plt.tight_layout()
    plt.show()

plot_series_and_variance(data, m=M)
```









What plots show?

From this high intensity plot we read:

- Data frequency is hourly
- Mainline seasonality(on which i focused specificly) is daily
- Data is not homoscedastic since varience is instable.
- Likely there is complex seasonality involved

4. Seasonal & Subseries Decomposition

```
def subseries_plot_hourly_and_calendar(df: pd.DataFrame, data_col: str = "ds", value_col: str = "y"):
    d = df[[data_col, value_col]].copy()
    d[data_col] = pd.to_datetime(d[data_col])
    d["year"] = d[data_col].dt.year
    d["month"] = d[data_col].dt.month
    d["hour"] = d[data_col].dt.hour
    d["dow"] = d[data_col].dt.dayofweek

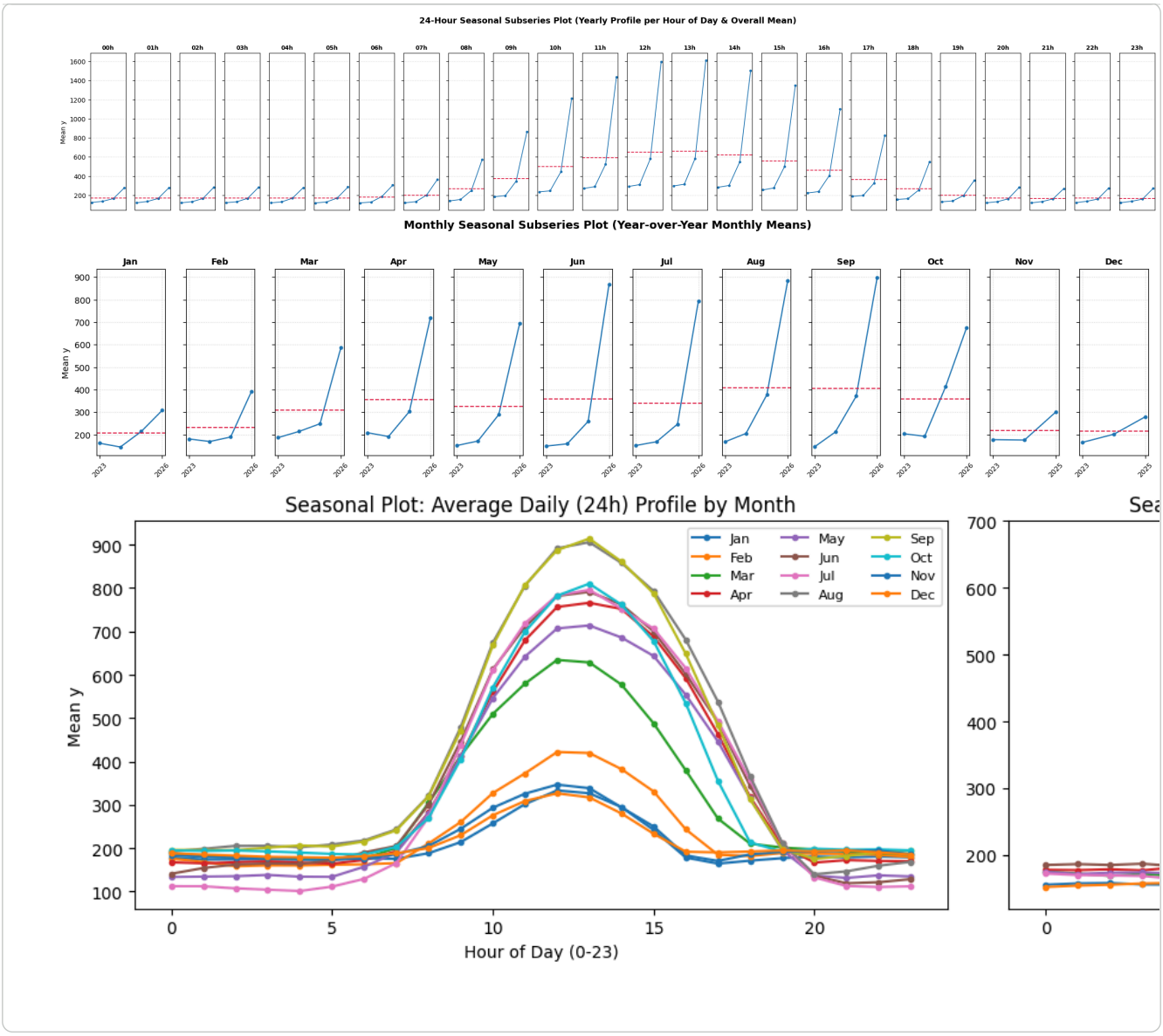
    fig, axes = plt.subplots(1, 24, figsize=(24, 4.2), sharey=True)
    hourly_year = d.groupby(["hour", "year"])[value_col].mean().reset_index()
    for hr in range(24):
        ax = axes[hr]
        sub = hourly_year[hourly_year["hour"] == hr].sort_values("year")
        overall_mean = d[d["hour"] == hr][value_col].mean()
        ax.plot(sub["year"], sub[value_col], color="#1f77b4", marker="o", markersize=3, linewidth=1.2)
        ax.axhline(overall_mean, color="crimson", linestyle="--", linewidth=1.2)
        ax.set_title(f"{hr:02d}h", fontsize=9, fontweight="bold")
        ax.set_xticks([])
        ax.grid(True, linestyle=":", alpha=0.5)
    axes[0].set_ylabel(f"Mean {value_col}")
    fig.suptitle("24-Hour Seasonal Subseries Plot (Yearly Profile per Hour of Day & Overall Mean)", fontsize=
    plt.tight_layout()
    plt.show()

    month_names = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
    fig, axes = plt.subplots(1, 12, figsize=(18, 4.2), sharey=True)
    monthly_year = d.groupby(["month", "year"])[value_col].mean().reset_index()
    for i, m_name in enumerate(month_names):
        ax = axes[i]
        sub = monthly_year[monthly_year["month"] == (i + 1)].sort_values("year")
        overall_mean = d[d["month"] == (i + 1)][value_col].mean()
        ax.plot(sub["year"], sub[value_col], color="#1f77b4", marker="o", markersize=3.5, linewidth=1.4)
        ax.axhline(overall_mean, color="crimson", linestyle="--", linewidth=1.2)
        ax.set_title(m_name, fontsize=10, fontweight="bold")
        if len(sub) > 1:
            ax.set_xticks([sub["year"].iloc[0], sub["year"].iloc[-1]])
        ax.tick_params(axis="x", rotation=45, labelsize=8)
        ax.grid(True, linestyle=":", alpha=0.5)
    axes[0].set_ylabel(f"Mean {value_col}")
    fig.suptitle("Monthly Seasonal Subseries Plot (Year-over-Year Monthly Means)", fontsize=13, fontweight="b
    plt.tight_layout()
    plt.show()

    fig, axes = plt.subplots(1, 2, figsize=(15, 4.2))
    hourly_month = d.groupby(["month", "hour"])[value_col].mean().unstack(0)
    for m_idx in hourly_month.columns:
        axes[0].plot(hourly_month.index, hourly_month[m_idx], marker="o", markersize=3, label=month_names[m_i
    axes[0].set_title("Seasonal Plot: Average Daily (24h) Profile by Month")
    axes[0].set_xlabel("Hour of Day (0-23)")
    axes[0].set_ylabel(f"Mean {value_col}")
    axes[0].legend(ncol=3, fontsize=8)

    dow_names = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
    hourly_dow = d.groupby(["dow", "hour"])[value_col].mean().unstack(0)
    for d_idx in hourly_dow.columns:
        axes[1].plot(hourly_dow.index, hourly_dow[d_idx], marker="o", markersize=3, label=dow_names[d_idx])
    axes[1].set_title("Seasonal Plot: Average Daily (24h) Profile by Day of Week")
    axes[1].set_xlabel("Hour of Day (0-23)")
    axes[1].legend(ncol=2, fontsize=8)
    plt.tight_layout()
    plt.show()

subseries_plot_hourly_and_calendar(data)
```



What we read from plots?

- No weeklty seasonality detected(thoughintuition said otherwise)
- Mounhthly seasonality with y being lowest at autumn-winter season
- growing y with years passing

5. Exogenous Weather Feature Correlation Analysis

```
def plot_feature_correlations(df: pd.DataFrame, m: int = 24, target_col: str = "y"):
    exog_cols = [c for c in df.select_dtypes(include=[np.number]).columns if c != target_col]
    corr_df = df[[target_col] + exog_cols].copy()

    corr_df["lag_1h"] = corr_df[target_col].shift(1)
    corr_df["lag_24h"] = corr_df[target_col].shift(24)
    corr_df["lag_168h"] = corr_df[target_col].shift(168)

    corr_matrix = corr_df.corr().round(3)
    print("=== PEARSON CORRELATION WITH TARGET 'y' ===")
    print(corr_matrix[target_col].sort_values(ascending=False).to_string())

    fig, ax = plt.subplots(figsize=(11, 7.5))
    sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1, ax=ax)
    ax.set_title("Feature & Lag Correlation Matrix")
    plt.tight_layout()
    plt.show()

    n_exog = len(exog_cols)
    n_cols = min(4, n_exog)
    n_rows = int(np.ceil(n_exog / n_cols))
```

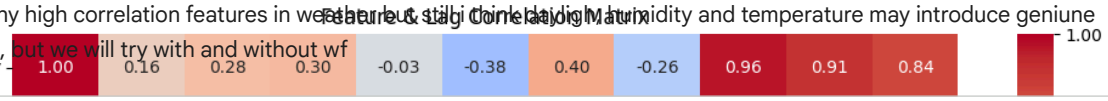
```
        fig, axes = plt.subplots(n_rows, n_cols, figsize=(4.2 * n_cols, 3.4 * n_rows))
        axes = np.atleast_1d(axes).ravel()
        for i, col in enumerate(exog_cols):
            ax = axes[i]
            sample = df[[col, target_col]].dropna()
            ax.scatter(sample[col], sample[target_col], alpha=0.05, s=6, color="#1f77b4")
            bins = pd.qcut(sample[col], q=15, duplicates="drop") if sample[col].nunique() > 15 else sample[col]
            binned = sample.groupby(bins, observed=False).agg(x=(col, "mean"), y=(target_col, "mean"))
            ax.plot(binned["x"], binned["y"], color="crimson", linewidth=2, marker="o", markersize=4, label="Binn
            r_val = sample[col].corr(sample[target_col])
            ax.set_title(f"{col} vs. {target_col} (r = {r_val:.3f})")
            ax.set_xlabel(col)
            ax.set_ylabel(target_col)
            ax.legend(fontsize=8)
        for j in range(len(exog_cols), len(axes)):
            axes[j].set_visible(False)
        plt.suptitle("Target y vs. Exogenous Weather Features", fontsize=13, fontweight="bold", y=1.02)
        plt.tight_layout()
        plt.show()

    plot_feature_correlations(data, m=M)
```

```
=== PEARSON CORRELATION WITH TARGET 'y' ===
y              1.000
lag_1h         0.964
lag_24h        0.905
lag_168h       0.836
daylight       0.395
wind_speed     0.303
temperature    0.280
cloud_cover    0.162
pressure      -0.025
air_density   -0.260
humidity      -0.380
```

From plots

- The only segment of the plot we care about really is y column
- don't see any high correlation features in weather but still think daylight, humidity and temperature may introduce geniune information, but we will try with and without wf




```
# ============================================================================
# ACF & PACF Analysis on Raw Undifferenced Series (y)
# ============================================================================
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.stats.diagnostic import acorr_ljungbox

def acf_pacf_raw(series: np.ndarray, m: int = 24, lags: int = 168, title: str = "Raw Undifferenced Series (y)
    z = np.asarray(series, float)
    z = z[~np.isnan(z)]
    max_lags = min(lags, len(z) // 2 - 1)

    fig, axes = plt.subplots(1, 2, figsize=(15, 4.2))
    plot_acf(z, lags=max_lags, zero=False, ax=axes[0], auto_ylims=True, title=f"{title} — ACF (lags=1..{max_l
    plot_pacf(z, lags=max_lags, zero=False, ax=axes[1], auto_ylims=True, title=f"{title} — PACF (lags=1..{max

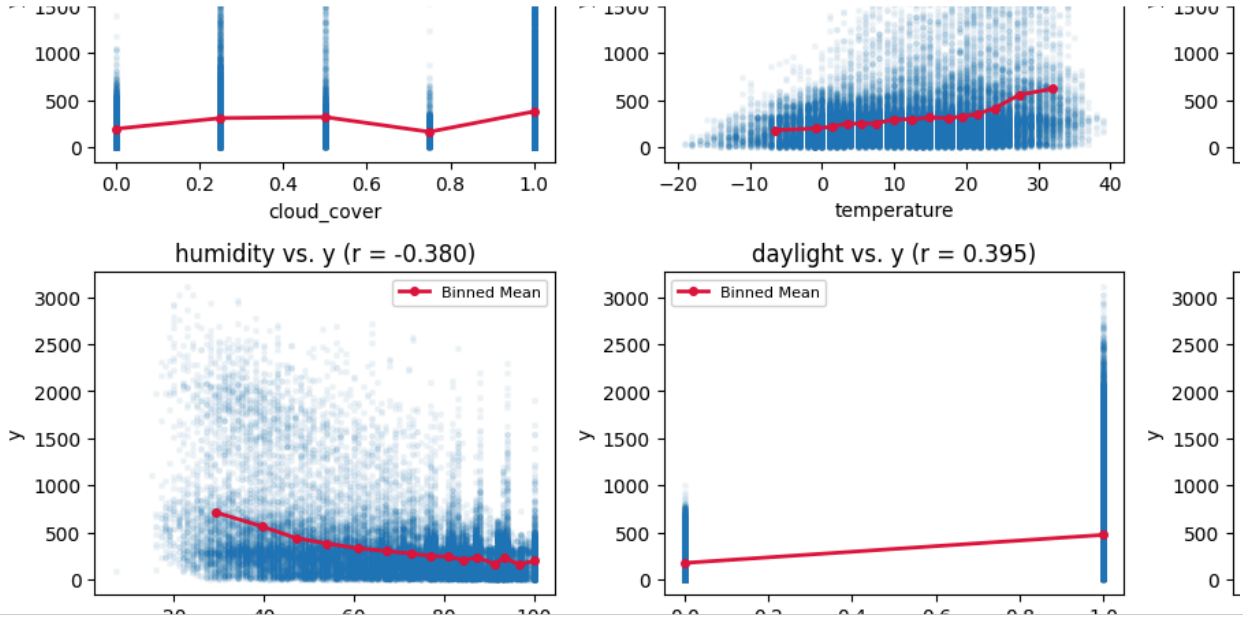
    # Highlight diurnal cycles (multiples of 24h) and weekly cycle (168h)
    for ax in axes:
        for k in range(m, max_lags + 1, m):
            color = "crimson" if k == 168 else "#e64173"
            alpha = 0.35 if k == 168 else 0.18
            lw = 3.5 if k == 168 else 2.5
            ax.axvline(k, color=color, alpha=alpha, linewidth=lw, linestyle="--")

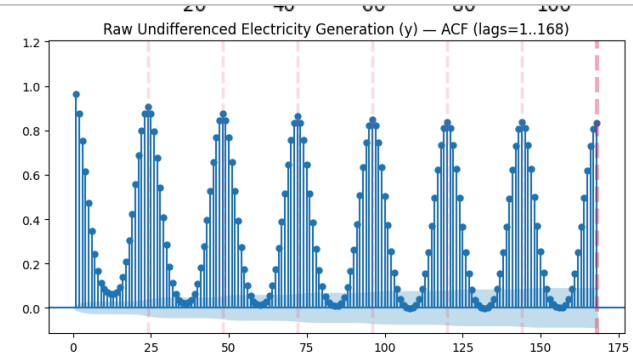
    plt.tight_layout()
    plt.show()

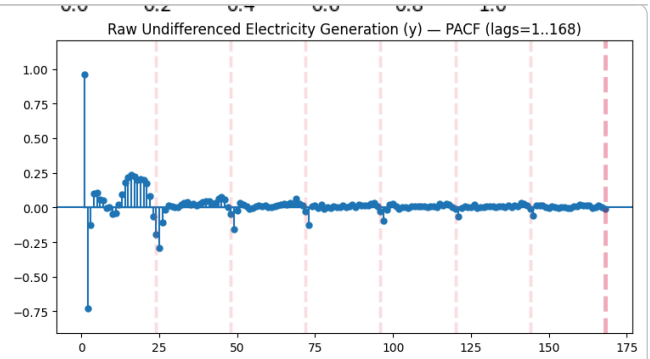
    # Numerical autocorrelation at key seasonal milestones
    s_series = pd.Series(z)
    key_lags = [1, 2, 3, 12, 24, 48, 72, 168]
    corr_vals = {f"lag_{k}": s_series.autocorr(k) for k in key_lags if k < len(z)}
    print("=== AUTOCORRELATION AT KEY SEASONAL LAGS (UNDIFFERENCED y) ===")
    for k_name, val in corr_vals.items():
        print(f"  {k_name:8s}: {val:+.4f}")

    # Ljung-Box test for raw series
    lb_raw = acorr_ljungbox(z, lags=[24, 48, 168], return_df=True)
    print("\nLjung-Box Test on Raw Undifferenced Series:")
    print(lb_raw.to_string())

# Run on raw series y across 7 daily cycles (168 hours = 1 week)
acf_pacf_raw(y, m=M, lags=7*M, title="Raw Undifferenced Electricity Generation (y)")
```








```
=== AUTOCORRELATION AT KEY SEASONAL LAGS (UNDIFFERENCED y) ===
  lag_1   : +0.9636
  lag_2   : +0.8765
  lag_3   : +0.7538
  lag_12  : +0.0621
  lag_24  : +0.9054
  lag_48  : +0.8758
  lag_72  : +0.8635
  lag_168 : +0.8358

Ljung-Box Test on Raw Undifferenced Series:
          lb_stat  lb_pvalue
24   2.166513e+05        0.0
48   4.017948e+05        0.0
168  1.236220e+06        0.0
```

From plots

- gradually descending seasonal(daily) acf and non-seasonal,however really high correlation, and rough pacf.

6. Stationarity Diagnostics & Differencing Plan

```
def acf_pacf(z: np.ndarray, m: int = 24, lags: int = 168, title: str = ""):
    z = np.asarray(z, float)
    z = z[~np.isnan(z)]
    lags = min(lags, len(z) // 2 - 1)
    fig, axes = plt.subplots(1, 2, figsize=(15, 4))
    plot_acf(z, lags=lags, zero=False, ax=axes[0], auto_ylims=True, title=f"{title} ACF (lags=1..{lags})".str
    plot_pacf(z, lags=lags, zero=False, ax=axes[1], auto_ylims=True, title=f"{title} PACF (lags=1..{lags})".s
    if m and m > 1:
        for ax in axes:
            for k in range(m, lags + 1, m):
                ax.axvline(k, color="#e64173", alpha=0.22, linewidth=4)
    plt.tight_layout()
    plt.show()


def kpss_test(series: np.ndarray, regression: str = "c", name: str = "Series") -> dict:
    clean = np.asarray(series, float)
    clean = clean[~np.isnan(clean)]
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        stat, p_value, n_lags, crit = kpss(clean, regression=regression, nlags="auto")
    is_stationary = stat < crit["5%"]
    status = "STATIONARY (fail to reject H0 at 5%)" if is_stationary else "NON-STATIONARY (reject H0 at 5%)"
    print(f"KPSS Test [{name}] (regression='{regression}'): stat={stat:.4f}, p-val~={p_value:.3f}, lags={n_la
    return {"name": name, "stat": stat, "p_value": p_value, "lags": n_lags, "crit_5%": crit["5%"], "stationar


def difference_plan(series: np.ndarray, m: int = 24, log: bool = False, dates: pd.Series | None = None, plot:
    x = np.log(series) if log else np.asarray(series, float)
    scale_label = "log(y)" if log else "y"

    print("=== STATIONARITY & DIFFERENCING PLAN ===")
    kpss_test(x, regression="c", name=f"Raw {scale_label} (level)")
    kpss_test(x, regression="ct", name=f"Raw {scale_label} (trend)")

    z_reg1 = np.diff(x, n=1)
    kpss_test(z_reg1, regression="c", name=f"First Regular Diff (d=1) on {scale_label}")

    D = nsdiffs(x, period=m)
    if D > 0:
```

```
            z_seas = x.copy()
            for _ in range(D):
                z_seas = z_seas[m:] - z_seas[:-m]
            kpss_test(z_seas, regression="c", name=f"Seasonal Diff (D={D}, lag={m}) on {scale_label}")
        else:
            z_s24 = x[m:] - x[:-m]
            kpss_test(z_s24, regression="c", name=f"Seasonal Diff (lag={m}) on {scale_label}")
            z_seas = x.copy()

        d = ndiffs(z_seas)
        z = np.diff(z_seas, n=d) if d else (z_seas if D > 0 else z_reg1)
        diff_desc = f"D={D}, d={d}" if (D > 0 or d > 0) else "d=1 (first difference)"
        if d > 0 and D > 0:
            kpss_test(z, regression="c", name=f"Combined Diff (D={D}, d={d}) on {scale_label}")

        print(f"Differencing summary: D={D}, d={d} ({len(x)} -> {len(z)} obs)")

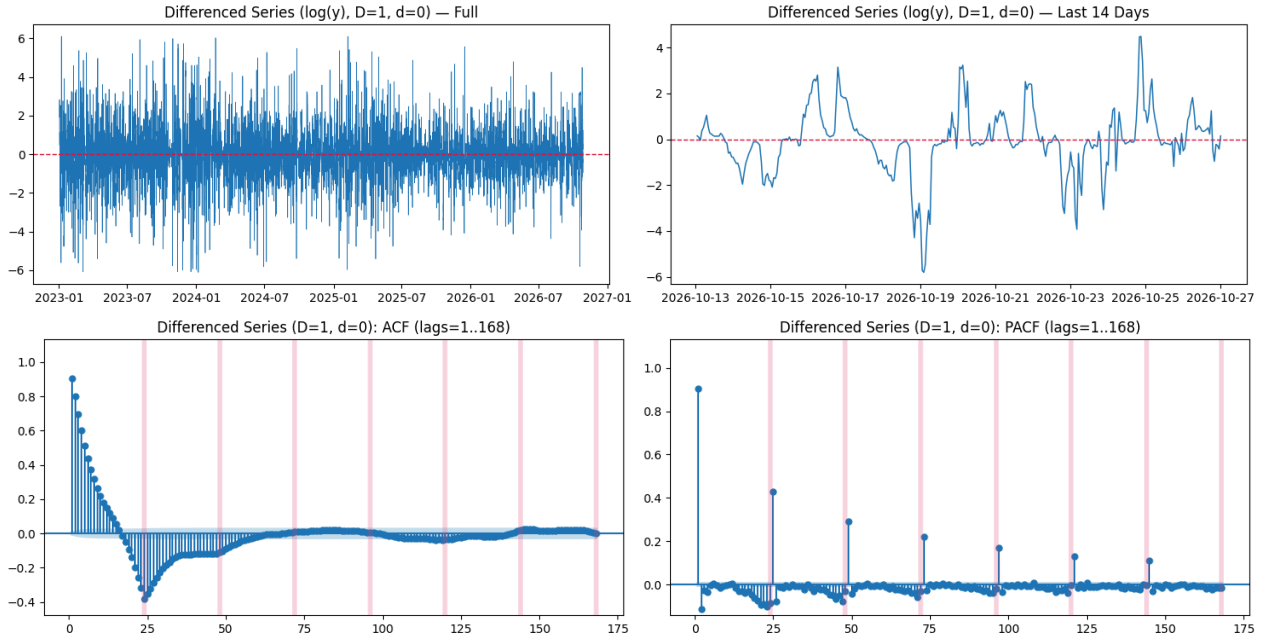
        if plot:
            zoom_h = min(len(z), 14 * m)
            fig, axes = plt.subplots(1, 2, figsize=(15, 3.8))
            d_index = pd.to_datetime(dates).iloc[len(x) - len(z):] if dates is not None else pd.RangeIndex(len(z)
            axes[0].plot(d_index, z, color="#1f77b4", linewidth=0.5)
            axes[0].axhline(0, color="crimson", linestyle="--", linewidth=1)
            axes[0].set_title(f"Differenced Series ({scale_label}, {diff_desc}) — Full")

            axes[1].plot(d_index[-zoom_h:], z[-zoom_h:], color="#1f77b4", linewidth=1.1)
            axes[1].axhline(0, color="crimson", linestyle="--", linewidth=1)
            axes[1].set_title(f"Differenced Series ({scale_label}, {diff_desc}) — Last 14 Days")
            plt.tight_layout()
            plt.show()

            acf_pacf(z, m=m, lags=7 * m, title=f"Differenced Series ({diff_desc}):")

difference_plan(y, m=M, log= True, dates=dates)
```

```
=== STATIONARITY & DIFFERENCING PLAN ===
KPSS Test [Raw log(y) (level)] (regression='c'): stat=19.2273, p-val~=0.010, lags=88, 5% crit=0.463 -> NON-STA
KPSS Test [Raw log(y) (trend)] (regression='ct'): stat=1.4302, p-val~=0.010, lags=78, 5% crit=0.146 -> NON-STA
KPSS Test [First Regular Diff (d=1) on log(y)] (regression='c'): stat=0.0068, p-val~=0.100, lags=227, 5% crit=
KPSS Test [Seasonal Diff (D=1, lag=24) on log(y)] (regression='c'): stat=0.0020, p-val~=0.100, lags=91, 5% cri
Differencing summary: D=1, d=0 (33480 -> 33456 obs)
```



From plots

- By default non stationary, even when log applied
- Enough one seasonal differencing(or regular) to provide stationarity.

7. STL & Multiseasonal MSTL Decomposition & Residual Diagnostics

```
# ============================================================================
# Full-Data Decomposition: STL (24h) & Multi-Seasonal MSTL (Daily 24h + Yearly 8760h)
# ============================================================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats
from statsmodels.tsa.seasonal import STL, MSTL as StatsmodelsMSTL
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.stats.diagnostic import acorr_ljungbox

def strengths(stl_fit) -> tuple[float, float]:
    resid = np.asarray(stl_fit.resid)
    trend = np.asarray(stl_fit.trend)
    seasonal = np.asarray(stl_fit.seasonal)
    seasonal_sum = seasonal.sum(axis=1) if seasonal.ndim == 2 else seasonal
    f_t = max(0.0, 1.0 - np.nanvar(resid) / np.nanvar(trend + resid))
    f_s = max(0.0, 1.0 - np.nanvar(resid) / np.nanvar(seasonal_sum + resid))
    return float(f_t), float(f_s)


def residual_diagnostics(resid: np.ndarray, dates: pd.Series, m: int = 24, lags: int = 168, label: str = "STL
    r = pd.Series(resid).dropna().to_numpy(float)
    d_valid = pd.to_datetime(dates).iloc[-len(r):]

    fig, axes = plt.subplots(2, 2, figsize=(15, 7))
    axes[0, 0].plot(d_valid, r, color="#1f77b4", linewidth=0.6)
    axes[0, 0].axhline(0, color="crimson", linestyle="--", linewidth=1)
    axes[0, 0].set_title(f"{label} Residuals (mean = {np.mean(r):.4f}, std = {np.std(r):.2f})")

    sns.histplot(r, bins=60, kde=True, ax=axes[0, 1], color="#1f77b4")
    axes[0, 1].set_title(f"{label} Residuals Distribution")

    stats.probplot(r, dist="norm", plot=axes[1, 0])
    axes[1, 0].set_title("Normal Q-Q Plot")

    plot_acf(r, lags=lags, zero=False, ax=axes[1, 1], auto_ylims=True, title=f"ACF of {label} Residuals (lags
    if m and m > 1:
        for k in range(m, lags + 1, m):
            axes[1, 1].axvline(k, color="#e64173", alpha=0.22, linewidth=4)

    plt.tight_layout()
    plt.show()

    lb_lags = [m, 2 * m, 7 * m] if m == 24 else [m, 2 * m]
    lb = acorr_ljungbox(r, lags=lb_lags, return_df=True)
    ks_stat, ks_pval = stats.kstest(r, "norm", args=(np.mean(r), np.std(r)))
    acf_m = pd.Series(r).autocorr(m)
    print(f"--- {label} Residual Diagnostics ---")
    print(f"Residual ACF(lag={m}) = {acf_m:.4f} | KS Normality stat = {ks_stat:.4f}, p-val = {ks_pval:.3e}")
    print("Ljung-Box test on residuals:")
    print(lb.to_string())
    print()


def decompose_and_analyze(
    series: np.ndarray,
    dates: pd.Series,
    m: int = 24,
    seasonal_window: int = 13,
    robust: bool = False,
    multiseasonal_periods: list[int] | None = [24, 8760],
):
    y_arr = np.asarray(series, float)
    ds_arr = pd.to_datetime(dates)

    # 1. STL (Daily period m=24)
    stl_fit = STL(y_arr, period=int(m), seasonal=int(seasonal_window), robust=robust).fit()
    f_t, f_s = strengths(stl_fit)
    print(f"=== STL DECOMPOSITION (period={m}, seasonal={seasonal_window}) ===")
    print(f"Trend Strength F_T = {f_t:.4f} | Seasonal Strength F_S (m={m}) = {f_s:.4f}")

    fig, axes = plt.subplots(4, 1, figsize=(15, 9), sharex=True)
    axes[0].plot(ds_arr, y_arr, color="black", linewidth=0.7, label="Observed")
    axes[0].plot(ds_arr, (y_arr - stl_fit.seasonal), color="#e64173", linewidth=0.8, alpha=0.8, label="Seasor
    axes[0].set_title(f"STL Full History: Observed vs. Seasonally Adjusted ({len(y_arr)} hours)")
    axes[0].legend(loc="upper right")
    axes[1].plot(ds_arr, stl_fit.trend, color="#1f77b4", linewidth=1.2)
```

```
        axes[1].set_title(f"STL Trend Component (F_T = {f_t:.3f})")
        axes[2].plot(ds_arr, stl_fit.seasonal, color="#2ca02c", linewidth=0.7)
        axes[2].set_title(f"STL Daily Seasonal Component (period={m}h, F_S = {f_s:.3f})")
        axes[3].plot(ds_arr, stl_fit.resid, color="gray", linewidth=0.6)
        axes[3].axhline(0, color="crimson", linestyle="--", linewidth=1)
        axes[3].set_title("STL Remainder / Residuals")
        plt.tight_layout()
        plt.show()

        residual_diagnostics(stl_fit.resid, ds_arr, m=m, lags=7 * m, label="STL")

        # 2. Multi-Seasonal MSTL (Daily m=24 + Yearly m=8760)
        if multiseasonal_periods is not None and len(multiseasonal_periods) > 1:
            print(f"=== MULTISEASONAL STL (MSTL) DECOMPOSITION (periods={multiseasonal_periods}) ===")
            # iterate=1 ensures fast and clean convergence across 8,760 subseries
            mstl_fit = StatsmodelsMSTL(y_arr, periods=multiseasonal_periods, iterate=1).fit()
            mf_t, mf_s = strengths(mstl_fit)
            print(f"MSTL Trend Strength F_T = {mf_t:.4f} | Combined Seasonal Strength F_S = {mf_s:.4f}")
            for idx, p in enumerate(multiseasonal_periods):
                s_col = mstl_fit.seasonal[:, idx] if isinstance(mstl_fit.seasonal, np.ndarray) else mstl_fit.seas
                f_sp = max(0.0, 1.0 - np.nanvar(mstl_fit.resid) / np.nanvar(s_col + mstl_fit.resid))
                p_label = "Daily" if p == 24 else ("Yearly" if p == 8760 else f"{p}h")
                print(f"  -> Individual Seasonal Strength F_S({p_label}, period={p}h) = {f_sp:.4f}")

            fig, axes = plt.subplots(2 + len(multiseasonal_periods), 1, figsize=(15, 2.5 * (2 + len(multiseasonal
            axes[0].plot(ds_arr, y_arr, color="black", linewidth=0.7, label="Observed")
            axes[0].plot(ds_arr, mstl_fit.trend, color="#1f77b4", linewidth=1.4, label="Trend")
            axes[0].set_title(f"MSTL Full History: Observed & Trend (F_T = {mf_t:.3f})")
            axes[0].legend(loc="upper right")

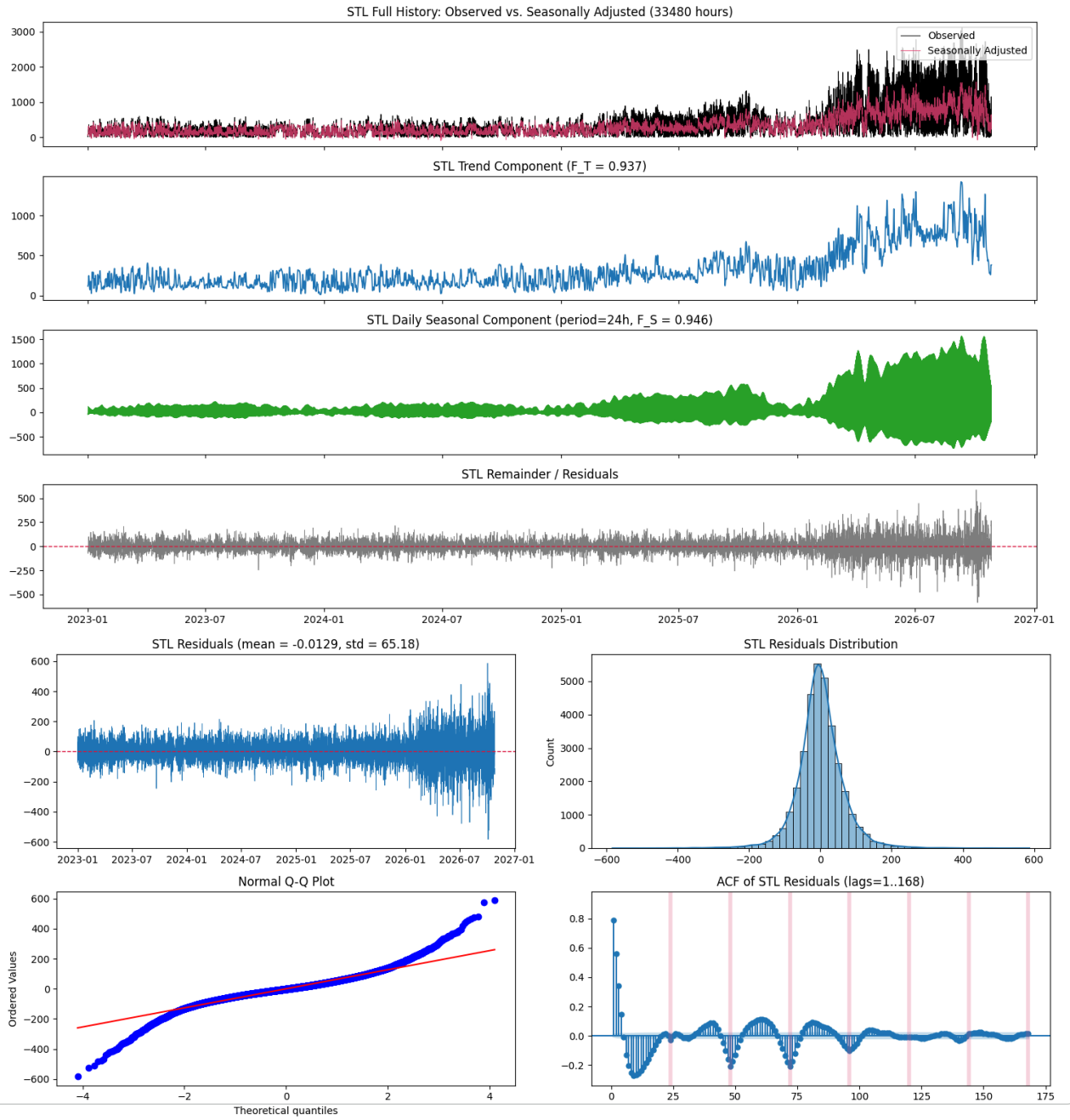
            for idx, p in enumerate(multiseasonal_periods):
                s_col = mstl_fit.seasonal[:, idx] if isinstance(mstl_fit.seasonal, np.ndarray) else mstl_fit.seas
                p_label = "Daily" if p == 24 else ("Yearly (Annual Wave)" if p == 8760 else f"{p}h")
                axes[1 + idx].plot(ds_arr, s_col, color="#2ca02c", linewidth=0.8)
                axes[1 + idx].set_title(f"MSTL {p_label} Seasonal Component (period={p}h, Full Data)")

            axes[-1].plot(ds_arr, mstl_fit.resid, color="gray", linewidth=0.6)
            axes[-1].axhline(0, color="crimson", linestyle="--", linewidth=1)
            axes[-1].set_title("MSTL Remainder / Residuals (Full Data)")
            plt.tight_layout()
            plt.show()

            residual_diagnostics(mstl_fit.resid, ds_arr, m=m, lags=7 * m, label="MSTL")

    # Execute on the entire dataset y across ~4 years with Daily (24h) and Yearly (8760h) seasons
    decompose_and_analyze(y, dates, m=M, multiseasonal_periods=[24, 8760])
```

```
=== STL DECOMPOSITION (period=24, seasonal=13) ===
Trend Strength F_T = 0.9371 | Seasonal Strength F_S (m=24) = 0.9464
```




```
--- STL Residual Diagnostics ---
Residual ACF(lag=24) = -0.0294 | KS Normality stat = 0.0593, p-val = 1.275e-102
Ljung-Box test on residuals:
     lb_stat     lb_pvalue
24  53590.454622        0.0
48  57750.027064        0.0
168 71917.872227        0.0
```

From plots

- IN both STL and MSTL still significant correlation presentin basically all lags. which points up to a rather poor performance than generalization residuals are not normal - have heavy tail from qqplot.

8. Evaluation Metrics, Benchmark & Conformal Prediction Intervals

```
=== MULTI-SEASONAL STL (MSTL) DECOMPOSITION (periods=[24, 168]) ===
MSTL Trend Strength F_T = 0.8333 | Combined Seasonal Strength F_S = 0.9232
  -> Individual Seasonal Strength F_S(Daily, period=24h) = 0.9078
```

```
SCALE = float(np.mean(np.abs(y[M:] - y[:-M])))


def mase(actual, forecast) -> float:
    return float(np.mean(np.abs(np.asarray(actual) - np.asarray(forecast))) / SCALE)


def rmse(actual, forecast) -> float:
    return float(np.sqrt(np.mean((np.asarray(actual) - np.asarray(forecast)) ** 2)))


def mape(actual, forecast) -> float:
    actual, forecast = np.asarray(actual, float), np.asarray(forecast, float)
    return float(np.mean(np.abs((actual - forecast) / np.where(np.abs(actual) < 1e-8, 1e-8, actual))) * 100)
```

```
def winkler(actual, lo, hi, alpha: float = ALPHA) -> float:
    actual, lo, hi = (np.asarray(a, float) for a in (actual, lo, hi))
    if np.isnan(lo).any() or np.isnan(hi).any():
        return np.nan
    return float(np.mean((hi - lo) + 2 / alpha * (np.clip(lo - actual, 0, None) + np.clip(actual - hi, 0, Nor


def coverage(actual, lo, hi) -> float:
    actual, lo, hi = (np.asarray(a, float) for a in (actual, lo, hi))
    if np.isnan(lo).any() or np.isnan(hi).any():
        return np.nan
    return float(np.mean((actual >= lo) & (actual <= hi)))


def conformal_radius(errors, level: float = LEVEL) -> float:
    return float(np.quantile(np.abs(np.asarray(errors)), level / 100))


def benchmark_forecasts(history: np.ndarray, h: int, m: int) -> dict[str, np.ndarray]:
    T = len(history)
    return {
        "mean":   np.repeat(history.mean(), h),
        "naive":  np.repeat(history[-1], h),
        "snaive": np.array([history[-m + (i % m)] for i in range(h)]),
        "drift":  history[-1] + np.arange(1, h + 1) * (history[-1] - history[0]) / (T - 1),
    }


def snaive_interval(history: np.ndarray, point: np.ndarray, h: int = H, m: int = M):
    history = np.asarray(history, float)
    sigma = np.std(history[m:] - history[:-m], ddof=1)
    half = Z_LEVEL * sigma * np.sqrt(np.arange(h) // m + 1)
    return np.clip(point - half, 0, None), point + half


train_split, valid_split = y[:-H], y[-H:]
simple_benchmarks = benchmark_forecasts(train_split, H, M)
BENCHMARK = min(simple_benchmarks, key=lambda name: rmse(valid_split, simple_benchmarks[name]))
print(f"Benchmark: {BENCHMARK} (Validation RMSE: {rmse(valid_split, simple_benchmarks[BENCHMARK]):.2f}, MASE:
```

```
          lb_stat  lb_pvalue
24   296793.483514        0.0
48   362549.675454        0.0
168  417063.684242        0.0
```

```
Benchmark: snaive (Validation RMSE: 120.40, MASE: 0.823)
```

What was found?

- Series are heteroscedastic and non-stationary, withseasonal swing increasing
- Hourly and monthly seasonality. Initially MSTL+ETS ran on weekly + seasonalitywhich could also bean overlook by me.
- MSTL and STL decomposition left huge correlation in resifuals, which was intiallyoverlooked by me.
- found some correlation in weather, which could bring us valuable patterns. especially daytime and humidity.

A plan

- Try every method
- Use log transform to stabilize invarience(If required)
- validate through backtest with rolling windows capturing mape, rmse winkler score and coverage
- Initially wanted to try MSTL + ETS or MSTL + ARIMA - big mistake.

9. Forecasters & Weather Regression Infrastructure

```
NAN = lambda h: np.full(h, np.nan)
_pretrained = {}
CHRONOS_CONTEXT = 8192


def _np(a):
    return a.detach().cpu().numpy() if hasattr(a, "detach") else np.asarray(a)


def exog_for(n_history: int, h: int):
    return X_ALL[:n_history], X_ALL[n_history:n_history + h], DS_ALL[n_history:n_history + h]
```

```
def weather_regression(x, X_hist, X_fut):
    mu, sd = X_hist.mean(0), X_hist.std(0) + 1e-9
    Z  = np.column_stack([np.ones(len(X_hist)), (X_hist - mu) / sd])
    Zf = np.column_stack([np.ones(len(X_fut)),  (X_fut  - mu) / sd])
    beta = np.linalg.solve(Z.T @ Z + 1e-3 * np.eye(Z.shape[1]), Z.T @ np.asarray(x, float))
    return Z @ beta, Zf @ beta


class Fitted:
    def __init__(self, model, log, X_fut=None, reg_future=None):
        self.model, self.log, self.X_fut, self.reg_future = model, log, X_fut, reg_future

    def predict(self, h: int, level=None):
        kw = {"X": self.X_fut[:h]} if self.X_fut is not None else {}
        out = dict(self.model.predict(h=h, level=level, **kw))
        if self.reg_future is not None:
            out = {k: v + self.reg_future[:h] for k, v in out.items()}
        return {k: np.exp(v) for k, v in out.items()} if self.log else out


def _window(x, train_window):
    return x[-train_window:] if (train_window is not None and len(x) > train_window) else x


def fit_ets(history, spec=None, damped=None, log=False, train_window=60 * 24,
            weather=False, X_hist=None, X_fut=None) -> Fitted:
    x = np.log(history) if log else np.asarray(history, float)
    reg_future = None
    if weather:
        reg_fit, reg_future = weather_regression(x, X_hist, X_fut)
        x = x - reg_fit
    x = _window(x, train_window)
    if spec is None:
        model = AutoETS(season_length=M, model="ZZZ" if not weather else "AZZ")
    else:
        spec = str(spec).strip().upper()
        if spec[2:4] in ("DA", "DM", "DN"):
            spec, damped = spec[:2] + spec[3:], True
        if weather:
            spec = spec.replace("M", "A")
        model = AutoETS(season_length=M, model=spec, damped=bool(damped) if spec[1] != "N" else None)
    return Fitted(model.fit(y=x), log, reg_future=reg_future)


def fit_arima(history, order=None, seasonal=(0, 0, 0), log=False, drift=False, train_window=60 * 24,
              weather=False, X_hist=None, X_fut=None) -> Fitted:
    x = np.log(history) if log else np.asarray(history, float)
    if order is None:
        x = _window(x, train_window)
        model = AutoARIMA(season_length=M, approximation=True)
    else:
        model = ARIMA(order=tuple(order), seasonal_order=tuple(seasonal), season_length=M,
                      include_drift=drift and (order[1] + seasonal[1]) == 1)
    if not weather:
        return Fitted(model.fit(y=x), log)
    Xh = X_hist[-len(x):]
    keep = Xh.std(0) > 1e-8
    return Fitted(model.fit(y=x, X=Xh[:, keep]), log, X_fut=X_fut[:, keep])


def fit_mstl(history, log=False, train_window=None, weather=False, X_hist=None, X_fut=None) -> Fitted:
    x = np.log(history) if log else np.asarray(history, float)
    reg_future = None
    if weather:
        reg_fit, reg_future = weather_regression(x, X_hist, X_fut)
        x = x - reg_fit
    return Fitted(MSTL(season_length=M).fit(y=_window(x, train_window)), log, reg_future=reg_future)


FITTERS = {"ets": fit_ets, "arima": fit_arima, "mstl": fit_mstl}


def fit_member(member, history, h: int = H, weather: bool = False) -> Fitted:
    kind, args = member
    args = dict(args)
    use_w = args.pop("weather", weather)
    if use_w:
        X_hist, X_fut, _ = exog_for(len(history), h)
        args.update(weather=True, X_hist=X_hist, X_fut=X_fut)
    else:
```

```
        args["weather"] = False
    return FITTERS[kind](history, **args)


def classic(member, weather: bool = False):
    def forecast(history, h):
        out = fit_member(member, history, h, weather=weather).predict(h=h, level=[LEVEL])
        return {"mean": out["mean"], LO: np.clip(out[LO], 0, None), HI: out[HI]}
    return forecast


def neural(model_class, input_size=None, max_steps=500, seed=42, weather: bool = False, **settings):
    from neuralforecast import NeuralForecast

    def forecast(history, h):
        use_w = weather and getattr(model_class, "EXOGENOUS_FUTR", False)
        df = pd.DataFrame({"unique_id": 1, "ds": DS_ALL[:len(history)], "y": history})
        extra, futr_df = {}, None
        if use_w:
            X_hist, X_fut, ds_fut = exog_for(len(history), h)
            df[EXOG] = X_hist
            futr_df = pd.DataFrame({"unique_id": 1, "ds": ds_fut})
            futr_df[EXOG] = X_fut
            extra["futr_exog_list"] = EXOG
        model = model_class(h=h, input_size=input_size or 3 * M, max_steps=max_steps, scaler_type="robust",
                            random_seed=seed, accelerator="gpu" if DEVICE == "cuda" else "cpu", devices=1,
                            enable_progress_bar=False, enable_model_summary=False, logger=False,
                            **extra, **settings)
        nf = NeuralForecast(models=[model], freq="H")
        nf.fit(df=df)
        pred = nf.predict(futr_df=futr_df) if use_w else nf.predict()
        return {"mean": pred[model_class.__name__].to_numpy(), LO: NAN(h), HI: NAN(h)}
    return forecast


def pretrained(name):
    if name not in _pretrained:
        if name == "Chronos-2":
            from chronos import BaseChronosPipeline
            _pretrained[name] = BaseChronosPipeline.from_pretrained("amazon/chronos-2", device_map=DEVICE)
        elif name == "TimesFM 2.5":
            import timesfm
            model = timesfm.TimesFM_2p5_200M_torch.from_pretrained("google/timesfm-2.5-200m-pytorch")
            model.compile(timesfm.ForecastConfig(max_context=512, max_horizon=128, normalize_inputs=True,
                                                 use_continuous_quantile_head=True, fix_quantile_crossing=Tru
            _pretrained[name] = model
        elif name == "TiRex":
          import sys
          # If the wrong 'tirex' IR package is cached in memory, purge it:
          if "tirex" in sys.modules and not hasattr(sys.modules["tirex"], "load_model"):
              del sys.modules["tirex"]
          try:
              from tirex import load_model
          except (ImportError, AttributeError):
              import subprocess
              subprocess.run([sys.executable, "-m", "pip", "uninstall", "-y", "tirex"], check=False)
              subprocess.run([sys.executable, "-m", "pip", "install", "-q", "tirex-ts==1.4.2"], check=False)
              if "tirex" in sys.modules:
                  del sys.modules["tirex"]
              from tirex import load_model
          _pretrained[name] = load_model("NX-AI/TiRex", device=DEVICE)
        else:
            raise ValueError(name)
    return _pretrained[name]


def foundation(name, weather: bool = False):
    def forecast(history, h):
        model = pretrained(name)
        hist = np.asarray(history, float)
        if weather:
            X_hist, X_fut, _ = exog_for(len(hist), h)

        if name == "Chronos-2":
            ctx = hist[-CHRONOS_CONTEXT:]
            levels = [ALPHA / 2, 0.5, 1 - ALPHA / 2]
            inp = torch.tensor(ctx, dtype=torch.float32)
            if weather:
                Xc = X_hist[-len(ctx):]
                inp = {"target": ctx,
```

```
                        "past_covariates":   {c: Xc[:, j] for j, c in enumerate(EXOG)},
                        "future_covariates": {c: X_fut[:, j] for j, c in enumerate(EXOG)}}
            try:
                q, _ = model.predict_quantiles([inp], prediction_length=h, quantile_levels=levels)
            except Exception:
                q, _ = model.predict_quantiles([torch.tensor(ctx, dtype=torch.float32)],
                                               prediction_length=h, quantile_levels=levels)
            q = _np(q[0]).reshape(-1, h, 3)[0]
            mean, lo, hi = q[:, 1], q[:, 0], q[:, 2]
        else:
            x, reg_future = hist, None
            if weather:
                reg_fit, reg_future = weather_regression(hist, X_hist, X_fut)
                x = hist - reg_fit
            context = torch.tensor(x, dtype=torch.float32)
            if name == "TimesFM 2.5":
                _, q = model.forecast(horizon=h, inputs=[context.numpy()])
                q = _np(q)[0, :h]
                mean, q10, q90 = q[:, 5], q[:, 1], q[:, 9]
            elif name == "TiRex":
                q, _ = model.forecast(context=[context], prediction_length=h)
                q = _np(q)[0]
                mean, q10, q90 = q[:, 4], q[:, 0], q[:, 8]
            else:
                raise ValueError(name)
            half = (q90 - q10) / 2 * Z_LEVEL / Z_80
            lo, hi = mean - half, mean + half
            if reg_future is not None:
                mean, lo, hi = mean + reg_future, lo + reg_future, hi + reg_future
        return {"mean": mean, LO: np.clip(lo, 0, None), HI: hi}
    return forecast
```

```
WINDOWS = 4
FORECASTERS = {}
MIXES = {}


def backtest(forecasters: dict, history: np.ndarray = y, h: int = H, windows: int = WINDOWS) -> pd.DataFrame:
    FORECASTERS.update(forecasters)
    frames, seconds = [], {}
    for k in range(windows, 0, -1):
        end = len(history) - (k - 1) * h
        past, actual = history[:end - h], history[end - h:end]
        bm = benchmark_forecasts(past, h, M)[BENCHMARK]
        bm_lo, bm_hi = snaive_interval(past, bm, h, M)
        frame = pd.DataFrame({"window": k, "step": np.arange(1, h + 1), "actual": actual,
                              "benchmark": bm, "benchmark lo": bm_lo, "benchmark hi": bm_hi})
        for name, f in forecasters.items():
            start = time.time()
            fc = f(past, h)
            seconds[name] = seconds.get(name, 0.0) + time.time() - start
            frame[name], frame[f"{name} lo"], frame[f"{name} hi"] = fc["mean"], fc[LO], fc[HI]
        frames.append(frame)
    bt = pd.concat(frames, ignore_index=True)
    bt.attrs["seconds"] = seconds
    return bt


def add(bt: pd.DataFrame, new: pd.DataFrame) -> pd.DataFrame:
    keep = ("window", "step", "actual", "benchmark", "benchmark lo", "benchmark hi")
    out = bt.drop(columns=[c for c in new.columns if c in bt.columns and c not in keep])
    cols = [c for c in new.columns if c not in out.columns]
    out = pd.concat([out, new[cols]], axis=1)
    out.attrs["seconds"] = {**bt.attrs.get("seconds", {}), **new.attrs.get("seconds", {})}
    return out


def combine(bt: pd.DataFrame, names: list[str], label: str) -> None:
    MIXES[label] = list(names)
    bt[label] = bt[names].mean(axis=1)
    bt[f"{label} lo"] = bt[[f"{n} lo" for n in names]].mean(axis=1, skipna=False)
    bt[f"{label} hi"] = bt[[f"{n} hi" for n in names]].mean(axis=1, skipna=False)


def forecasts_in(bt: pd.DataFrame) -> list[str]:
    names = [c for c in bt.columns if c not in ("window", "step", "actual", "benchmark")
             and not c.endswith((" lo", " hi"))]
    return names + ["benchmark"]
```

```
def _interval(bt, g, c, w):
    if f"{c} lo" in g.columns and g[f"{c} lo"].notna().all():
        return g[f"{c} lo"].to_numpy(float), g[f"{c} hi"].to_numpy(float), "own"
    others = bt[bt["window"] != w]
    r = conformal_radius(others["actual"] - others[c])
    pred = g[c].to_numpy(float)
    return np.clip(pred - r, 0, None), pred + r, "conformal"


def window_table(bt: pd.DataFrame, columns: list[str] = None) -> pd.DataFrame:
    columns = columns or forecasts_in(bt)
    wins = sorted(bt["window"].unique(), reverse=True)
    label = {w: f"window {w}" + (" (validation)" if w == 1 else "") for w in wins}
    rows, best = {c: {} for c in columns}, {c: 0 for c in columns}
    for w in wins:
        g = bt[bt["window"] == w]
        actual = g["actual"].to_numpy(float)
        scores = {}
        for c in columns:
            lo, hi, kind = _interval(bt, g, c, w)
            scores[c] = rmse(actual, g[c])
            rows[c].setdefault("rmse", {})[w] = scores[c]
            rows[c].setdefault("mape", {})[w] = mape(actual, g[c])
            rows[c].setdefault("winkler", {})[w] = winkler(actual, lo, hi)
            rows[c].setdefault("coverage", {})[w] = 100 * coverage(actual, lo, hi)
            rows[c]["interval"] = kind
        best[min(columns, key=scores.get)] += 1

    table = pd.DataFrame({label[w]: [rows[c]["rmse"][w] for c in columns] for w in wins}, index=columns)
    table["mean"] = table.mean(axis=1)
    table["mean rmse"] = table["mean"]
    table["mean mape (%)"] = [np.mean(list(rows[c]["mape"].values())) for c in columns]
    table[f"mean winkler ({LEVEL}%)"] = [np.mean(list(rows[c]["winkler"].values())) for c in columns]
    table[f"mean coverage ({LEVEL}%)"] = [np.mean(list(rows[c]["coverage"].values())) for c in columns]
    table["interval"] = [rows[c]["interval"] for c in columns]
    table["windows won"] = [best[c] for c in columns]
    table["seconds"] = [bt.attrs.get("seconds", {}).get(c, np.nan) for c in columns]
    return table


def _from_all_of_y(label: str) -> dict:
    if label in MIXES:
        sub = [_from_all_of_y(part) for part in MIXES[label]]
        return {"mean": np.mean([p["mean"] for p in sub], axis=0),
                LO: np.mean([p[LO] for p in sub], axis=0),
                HI: np.mean([p[HI] for p in sub], axis=0)}
    if label == "benchmark":
        point = benchmark_forecasts(y, H, M)[BENCHMARK]
        lo, hi = snaive_interval(y, point)
        return {"mean": point, LO: lo, HI: hi}
    return FORECASTERS[label](y, H)


def final_forecast(bt: pd.DataFrame, label: str, interval: str = "conformal"):
    fc = _from_all_of_y(label)
    if interval == "conformal":
        r = conformal_radius(bt["actual"] - bt[label])
        return fc["mean"], np.clip(fc["mean"] - r, 0, None), fc["mean"] + r
    if interval == "own interval":
        if np.isnan(fc[LO]).any():
            raise ValueError(f"{label} has no interval of its own; use INTERVAL = 'conformal'")
        return fc["mean"], fc[LO], fc[HI]
    raise ValueError(interval)
```

```
!pip uninstall -y tirex 2>/dev/null
!pip install -q statsmodels statsforecast==2.1.1 neuralforecast==3.3.0 chronos-forecasting==2.3.2 "timesfm[tc
```

How to validate

- Basically utilised framework fromlabs so wee have 4 rolling windows with backtest on which we gather rmse on all windows, mean mape, mean winkler score and mean coverage
- After session 1 i ran all models with weather on and weather off.

10. Validation Run 1: Models Without Weather

```
from neuralforecast.models import LSTM, MLP, DeepAR, TiDE

CLASSIC_MEMBERS = {
    "AutoETS":   ("ets",   {"spec": None,  "log": True, "weather": False}),
    "AutoARIMA": ("arima", {"order": None, "log": True, "weather": False}),
    "MSTL":      ("mstl",  {"log": True,   "weather": False}),
}

CLASSIC_NOWEATHER = {name: classic(spec, weather=False) for name, spec in CLASSIC_MEMBERS.items()}
bt = backtest(CLASSIC_NOWEATHER)
```

```
NEURAL_NOWEATHER = {
    "MLP":    neural(MLP, weather=False),
    "DeepAR": neural(DeepAR, weather=False),
    "LSTM":   neural(LSTM, weather=False),
    "TiDE":   neural(TiDE, weather=False),
}
bt = add(bt, backtest(NEURAL_NOWEATHER))
```

```
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:lightning_fabric.utilities.seed:Seed set to 42
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped: `max_steps=500` reached.
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:lightning_fabric.utilities.seed:Seed set to 42
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped: `max_steps=500` reached.
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:lightning_fabric.utilities.seed:Seed set to 42
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped: `max_steps=500` reached.
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:lightning_fabric.utilities.seed:Seed set to 42
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped: `max_steps=500` reached.
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:lightning_fabric.utilities.seed:Seed set to 42
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped: `max_steps=500` reached.
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:lightning_fabric.utilities.seed:Seed set to 42
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped: `max_steps=500` reached.
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:lightning_fabric.utilities.seed:Seed set to 42
```

```
FOUNDATION_NOWEATHER = {name: foundation(name, weather=False) for name in ["Chronos-2", "TimesFM 2.5"]}
bt = add(bt, backtest(FOUNDATION_NOWEATHER))

table_noweather = window_table(bt, list(CLASSIC_MEMBERS) + list(NEURAL_NOWEATHER) + list(FOUNDATION_NOWEATHER
table_noweather.sort_values("mean")
```

```
Loading weights: 100%  170/170 [00:00<00:00, 841.05it/s]
```

	window 4	window 3	window 2	window 1 (validation)	mean	mean rmse	mean mape (%)	mean winkler (95%)	mean coverage (95%)
benchmark	282.722709	202.000491	180.970932	120.404317	196.524612	196.524612	667.069202	1063.741733	89.583333
TimesFM 2.5	194.118517	244.098606	274.359711	176.179943	222.189194	222.189194	318.608135	843.833693	100.000000
LSTM	527.287495	135.738306	100.335267	140.439085	225.950038	225.950038	1403.341749	3032.073275	76.562500
DeepAR	391.358077	228.390648	123.415037	164.708987	226.968187	226.968187	394.756858	1584.305892	93.750000
Chronos-2	257.432160	202.614123	334.322946	115.741767	227.527749	227.527749	435.696450	1147.018754	95.833333
TiDE	290.892758	344.824859	219.546601	134.734771	247.499747	247.499747	295.429762	1228.667083	93.750000
MLP	434.520168	270.840001	160.154840	176.585883	260.525223	260.525223	549.261574	1743.378620	89.583333
AutoARIMA	341.848655	442.584649	280.592855	330.406055	348.858054	348.858054	243.037765	2804.787370	79.687500
AutoETS	854.358255	597.438268	407.006869	400.930781	564.933544	564.933544	238.298741	10023.018725	80.729167
MSTL	796.721082	104.193944	758.458926	700.418085	589.948009	589.948009	250.622749	11915.686562	87.500000

Results

- Nobody beat benchmark in mean rmse - our main target, by which intial submission is scored.Only TimesFM 2.5 showed better results in winkler score, and was closest y rmse. in mape, almost every model was better .

11. Validation Run 2: Models With Weather

```
CLASSIC_WEATHER_MEMBERS = {
    "AutoETS (weather)":   ("ets",   {"spec": None,  "log": True, "weather": True}),
    "AutoARIMA (weather)": ("arima", {"order": None, "log": True, "weather": True}),
    "MSTL (weather)":      ("mstl",  {"log": True,   "weather": True}),
}
CLASSIC_WEATHER = {name: classic(spec, weather=True) for name, spec in CLASSIC_WEATHER_MEMBERS.items()}
bt = add(bt, backtest(CLASSIC_WEATHER))

NEURAL_WEATHER = {
    "MLP (weather)":    neural(MLP, weather=True),
    "DeepAR (weather)": neural(DeepAR, weather=True),
    "LSTM (weather)":   neural(LSTM, weather=True),
    "TiDE (weather)":   neural(TiDE, weather=True),
}
bt = add(bt, backtest(NEURAL_WEATHER))

FOUNDATION_WEATHER = {
    "Chronos-2 (weather)": foundation("Chronos-2", weather=True),
}
bt = add(bt, backtest(FOUNDATION_WEATHER))

table_weather = window_table(bt, list(CLASSIC_WEATHER_MEMBERS) + list(NEURAL_WEATHER) + list(FOUNDATION_WEATH
table_weather.sort_values("mean")
```

```
INFO:lightning_fabric.utilities.seed:Seed set to 42
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped: `max_steps=500` reached.
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
```

```
# ============================================================================
# MSTL with AutoETS on each decomposition component (bad attempt / baseline)
# ============================================================================
from statsmodels.tsa.seasonal import MSTL as StatsmodelsMSTL
from statsforecast.models import AutoETS

def mstl_per_component_ets(history: np.ndarray, h: int = H, periods: list[int] = [24, 168], train_window: int
    hist = np.asarray(history, float)
    if train_window is not None and len(hist) > train_window:
        hist = hist[-train_window:]

    # 1. Multi-seasonal MSTL decomposition
    decomp = StatsmodelsMSTL(hist, periods=periods).fit()

    # 2. Fit AutoETS on Trend (non-seasonal: season_length=1)
    f_trend = AutoETS(season_length=1, model="ZZZ").fit(y=decomp.trend).predict(h=h)["mean"]

    # 3. Fit AutoETS on Daily Seasonality (m=24) and repeat Weekly Seasonality (m=168)
    f_s24 = AutoETS(season_length=24, model="ANA").fit(y=decomp.seasonal[:, 0]).predict(h=h)["mean"]
    f_s168 = np.array([decomp.seasonal[-168 + (i % 168), 1] for i in range(h)])

    # 4. Fit AutoETS on Remainder / Residuals (Simple Exponential Smoothing: ANN)
    f_resid = AutoETS(season_length=1, model="ANN").fit(y=decomp.resid).predict(h=h)["mean"]

    # 5. Combined additive point forecast across all components
    point = np.clip(f_trend + f_s24 + f_s168 + f_resid, 0, None)
    return {"mean": point, LO: NAN(h), HI: NAN(h)}

# Register into backtest and score against standalone AutoETS and Benchmark
BAD_ATTEMPT_NAME = "MSTL (AutoETS on components, bad attempt)"
bt = add(bt, backtest({BAD_ATTEMPT_NAME: mstl_per_component_ets}))

table_bad_attempt = window_table(bt, [BAD_ATTEMPT_NAME, "AutoETS", "benchmark"])
table_bad_attempt
```

```
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 CUDA_VISIBLE_DEVICES: [0]
```

	window 4	window 3	window 2	window 1 (validation)	mean	mean rmse	mean mape (%)	mean winkler (95%)	mean coverage (95%)
	window_4	window_3	window_2	window_1	mean	mean	mean	winkler	coverage
MSTL (AutoETS on components, bad attempt)	271.597290	318.087669	579.637677	297.148460	366.617774	366.617774	920.703724	1956.445196	92.708333
LSTM (weather)	854.368255	597.438268	407.006869	400.930781	564.933544	564.933544	238.298741	10023.018725	80.729167
AutoETS	371.551428	199.849997	108.685987	136.666674	204.188372	204.188372	981.447523	1592.140054	89.062500
benchmark	282.722709	202.000491	180.970932	120.404317	196.524612	196.524612	667.069202	1063.741733	89.583333
AutoARIMA (weather)	501.423537	136.521458	144.361915	144.840865	231.786944	231.786944	402.978495	1806.881240	78.125000
TiDE (weather)	809.775288	344.517143	233.596826	132.697952	252.896802	252.896802	321.723317	1302.197487	92.708333
MLP (weather)	342.622885	257.367067	240.899547	207.297455	262.046738	262.046738	654.698667	1099.949383	95.312500
DeepAR (weather)	447.225973	509.161485	293.856636	204.008432	363.563132	363.563132	422.915579	1615.819033	93.750000
AutoETS (weather)	848.004153	561.248317	365.558958	296.909830	517.930314	517.930314	225.810779	11353.975163	85.937500
MSTL (weather)	814.179823	206.130588	353.308426	1024.354218	599.344689	599.344689	207.269819	18769.468149	90.104167

Next steps: [New interactive sheet]

Result of first approach

- Dirty, yes, flawed yes,but it's first try which was done in 1hour 30minutes,though scored more than on validation
- was trained on last 240 day due to time limit i faced.
- stillscored 20 place with adequate winkler score
- when i was validating on session 1 i wasonly focusing on the last window winkler score and mape, and i was kind of panicing i didn't thinkclealy because my limit on aargon which helped me code ended in last minute.

13. Validation Run: Multi-Family Combination [TimesFM 2.5, Chronos-2, MLP, DeepAR, AutoETS] Without Weather

```
COMBO_NOWEATHER_NAME = "TimesFM 2.5 + Chronos-2 + MLP + DeepAR + AutoETS (without weather)"
COMBO_NOWEATHER_PARTS = ["TimesFM 2.5", "Chronos-2", "MLP", "DeepAR", "AutoETS"]
combine(bt, COMBO_NOWEATHER_PARTS, COMBO_NOWEATHER_NAME)

table_combo_noweather = window_table(bt, COMBO_NOWEATHER_PARTS + [COMBO_NOWEATHER_NAME, "benchmark"])
table_combo_noweather.sort_values("mean")
```

	window 4	window 3	window 2	window 1 (validation)	mean	mean rmse	mean mape (%)	mean winkler (95%)	mean coverage (95%)
TimesFM 2.5 + Chronos-2 + MLP + DeepAR + AutoETS (without weather)	153.187070	122.730146	91.165063	142.215028	127.324327	127.324327	363.122258	656.160319	94.270833
benchmark	282.722709	202.000491	180.970932	120.404317	196.524612	196.524612	667.069202	1063.741733	89.583333
TimesFM 2.5	194.118517	244.098606	274.359711	176.179943	222.189194	222.189194	318.608135	843.833693	100.000000
DeepAR	391.358077	228.390648	123.415037	164.708987	226.968187	226.968187	394.756858	1584.305892	93.750000
Chronos-2	257.432160	202.614123	334.322946	115.741767	227.527749	227.527749	435.696450	1147.018754	95.833333
MLP	434.520168	270.840001	160.154840	176.585883	260.525223	260.525223	549.261574	1743.378620	89.583333
AutoETS	854.358255	597.438268	407.006869	400.930781	564.933544	564.933544	238.298741	10023.018725	80.729167

Results

initial combination showed significant increase beatingbenchmark dry,errors correlation showedminimal correlation between those models, so manual picking won the day

14. Validation Run: Ensemble [Chronos-2, LSTM, Benchmark] With Weather

```
COMBO_WEATHER_NAME = "Chronos-2 + LSTM+ benchmark (with weather)"
COMBO_WEATHER_PARTS = ["Chronos-2 (weather)", "benchmark", "LSTM (weather)"]
combine(bt, COMBO_WEATHER_PARTS, COMBO_WEATHER_NAME)

table_combo_weather = window_table(bt, COMBO_WEATHER_PARTS + [COMBO_WEATHER_NAME])
table_combo_weather.sort_values("mean")
```

	window 4	window 3	window 2	window 1 (validation)	mean	mean rmse	mean mape (%)	mean winkler (95%)	mean coverage (95%)
Chronos-2 + LSTM+ benchmark (with weather)	179.122568	141.911327	97.106870	91.701259	127.460506	127.460506	548.359533	596.832388	93.750000
Chronos-2 (weather)	208.098541	176.231642	86.274484	239.307338	177.478001	177.478001	270.177529	826.022598	100.000000
benchmark	282.722709	202.000491	180.970932	120.404317	196.524612	196.524612	667.069202	1063.741733	89.583333
LSTM (weather)	371.551428	199.849397	108.685987	136.666674	204.188372	204.188372	981.447523	1592.140054	89.062500

Results with weather

- Results somewhat similiar to the non-weather combinaion rmse-wise.

15. Comprehensive Scoreboard & Model Comparison

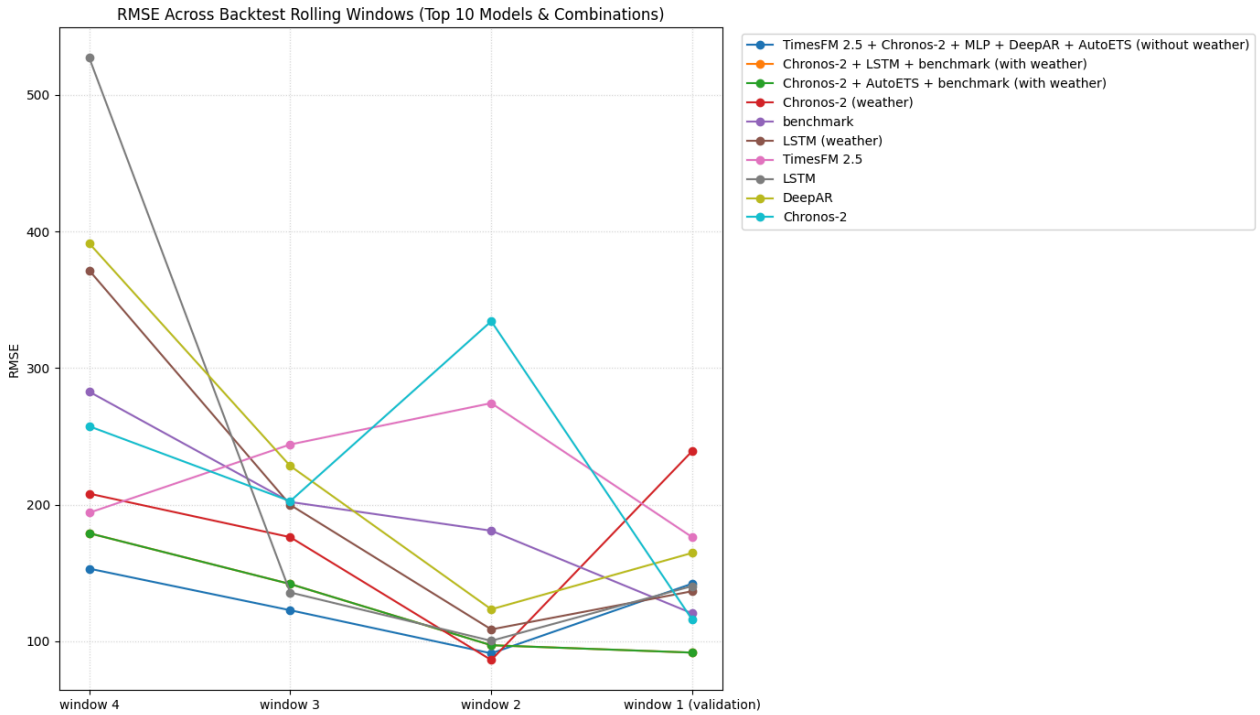
```
ALL_MODELS = forecasts_in(bt)
scoreboard = window_table(bt, ALL_MODELS)
scoreboard_sorted = scoreboard.sort_values("mean")
scoreboard_sorted
```

	window 4	window 3	window 2	window 1 (validation)	mean	mean rmse	mean mape (%)	mean winkler (95%)	mea coverag (95%
TimesFM 2.5 + Chronos-2 + MLP + DeepAR + AutoETS (without weather)	153.187070	122.730146	91.165063	142.215028	127.324327	127.324327	363.122258	656.160319	94.27083
Chronos-2 + LSTM + benchmark (with weather)	179.122568	141.911327	97.106870	91.701259	127.460506	127.460506	548.359533	596.832388	93.75000
Chronos-2 + AutoETS + benchmark (with weather)	179.122568	141.911327	97.106870	91.701259	127.460506	127.460506	548.359533	596.832388	93.75000
Chronos-2 (weather)	208.098541	176.231642	86.274484	239.307338	177.478001	177.478001	270.177529	826.022598	100.00000
benchmark	282.722709	202.000491	180.970932	120.404317	196.524612	196.524612	667.069202	1063.741733	89.58333
LSTM (weather)	371.551428	199.849397	108.685987	136.666674	204.188372	204.188372	981.447523	1592.140054	89.06250
TimesFM 2.5	194.118517	244.098606	274.359711	176.179943	222.189194	222.189194	318.608135	843.833693	100.00000
LSTM	527.287495	135.738306	100.335267	140.439085	225.950038	225.950038	1403.341749	3032.073275	76.56250
DeepAR	391.358077	228.390648	123.415037	164.708987	226.968187	226.968187	394.756858	1584.305892	93.75000
Chronos-2	257.432160	202.614123	334.322946	115.741767	227.527749	227.527749	435.696450	1147.018754	95.83333
AutoARIMA (weather)	501.423537	136.521458	144.361915	144.840865	231.786944	231.786944	402.978495	1806.881240	78.12500
TiDE	290.892758	344.824859	219.546601	134.734771	247.499747	247.499747	295.429762	1228.667083	93.75000
TiDE (weather)	300.775288	344.517143	233.596826	132.697952	252.896802	252.896802	321.723317	1302.197487	92.70833
MLP	434.520168	270.840001	160.154840	176.585883	260.525223	260.525223	549.261574	1743.378620	89.58333
MLP (weather)	342.622885	257.367067	240.899547	207.297455	262.046738	262.046738	654.698667	1099.949383	95.31250
AutoARIMA	341.848655	442.584649	280.592855	330.406055	348.858054	348.858054	243.037765	2804.787370	79.68750
DeepAR (weather)	447.225973	509.161485	293.856636	204.008432	363.563132	363.563132	422.915579	1615.819033	93.75000
MSTL (AutoETS on components, bad attempt)	271.597290	318.087669	579.637677	297.148460	366.617774	366.617774	920.703724	1956.445196	92.70833
MSTL + AutoETS (without weather, bad attempt)	825.533607	315.642427	191.532555	185.250179	379.489692	379.489692	226.838860	9949.956548	88.54166
AutoETS (weather)	848.004153	561.248317	365.558956	296.909830	517.930314	517.930314	225.810779	11353.975163	85.93750
AutoETS	854.358255	597.438268	407.006869	400.930781	564.933544	564.933544	238.298741	10023.018725	80.72916
MSTL	796.721082	104.193944	758.458926	700.418085	589.948009	589.948009	250.622749	11915.686562	87.50000
MSTL (weather)	814.179523	206.136588	353.508426	1024.354218	599.544689	599.544689	207.269819	18769.468149	90.10416

Next steps: New interactive sheet

```
window_cols = ["window 4", "window 3", "window 2", "window 1 (validation)"]
fig, ax = plt.subplots(figsize=(14, 8))
top_models = scoreboard_sorted.head(10).index
for m_name in top_models:
    ax.plot(window_cols, scoreboard_sorted.loc[m_name, window_cols], marker="o", linewidth=1.5, label=m_name)
ax.set_title("RMSE Across Backtest Rolling Windows (Top 10 Models & Combinations)")
ax.set_ylabel("RMSE")
```

```
ax.grid(True, linestyle=":", alpha=0.5)
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()
```



Andfor the submission i chose weather combination due to lower winkler score. I would like to try comboof weather and non-weather ensemble which on validation produced even higher score, but i'm too scacred and it's not much time left, and my laptop battery is about to be dead

16. Forecast Error Correlation Heatmap

```
# Forecast errors across all backtest windows: error = actual - forecast
eval_models = [c for c in forecasts_in(bt) if c in bt.columns]
error_df = pd.DataFrame({m: bt["actual"] - bt[m] for m in eval_models})
corr_errors = error_df.corr().round(2)

fig, ax = plt.subplots(figsize=(max(11, len(eval_models) * 0.75), max(9, len(eval_models) * 0.65)))
sns.heatmap(
    corr_errors,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
    linewidths=0.5,
    cbar_kws={"label": "Error Correlation (Pearson r)"},
    ax=ax
)
ax.set_title("Forecast Error Correlation Matrix Across All Rolling Windows", fontsize=13, fontweight="bold",
plt.xticks(rotation=45, ha="right", fontsize=9)
plt.yticks(rotation=0, fontsize=9)
plt.tight_layout()
plt.show()

# Focused heatmap of candidate ensemble members:
key_members = [m for m in ["benchmark", "LSTM", "LSTM (weather)", "Chronos-2", "Chronos-2 (weather)",
                           "TimesFM 2.5", "TiRex", "MLP", "MLP (weather)", "DeepAR", "DeepAR (weather)",
                           "AutoETS", "AutoARIMA", "MSTL"] if m in error_df.columns]
if len(key_members) > 1:
    fig, ax = plt.subplots(figsize=(10, 8))
    sns.heatmap(
        error_df[key_members].corr().round(2),
        annot=True,
        fmt=".2f",
        cmap="coolwarm",
```

```
            vmin=-1,
            vmax=1,
            linewidths=0.5,
            cbar_kws={"label": "Pearson r"},
            ax=ax
        )
        ax.set_title("Forecast Error Correlation: Key Individual Models & Ensemble Components", fontsize=12, font
        plt.xticks(rotation=45, ha="right", fontsize=9)
        plt.yticks(rotation=0, fontsize=9)
        plt.tight_layout()
        plt.show()

    corr_errors
```